

Reporte de Práctica: Modularización Avanzada y Pasaje de Matrices a Funciones

Asignatura:	Programación Estructurada
Estudiante:	Aldo Ezequiel Rodriguez Godoy
Código:	224097617
Sección:	D04
Profesor:	JORGE ERNESTO LOPEZ ARCE DELGADO
Fecha:	20 de marzo del 2026

Introducción

La programación estructurada se basa en el principio fundamental de "divide y vencerás". A medida que los sistemas de software crecen en complejidad, mantener la totalidad de la lógica algorítmica dentro de un único bloque ejecutable como la función principal `main` se vuelve inviable, propenso a errores y sumamente difícil de mantener. La modularización mediante funciones permite fragmentar un problema complejo en subproblemas o subrutinas independientes, promoviendo la reutilización de código, el encapsulamiento y la legibilidad del sistema.

En el lenguaje C, el pasaje de estructuras de datos bidimensionales (matrices) a funciones independientes introduce una restricción sintáctica y arquitectónica crítica. A diferencia de las variables primitivas o los arreglos unidimensionales, al transferir una matriz como parámetro, el compilador requiere obligatoriamente conocer el tamaño de la segunda dimensión (las columnas). Esto se debe a la forma en que la memoria RAM gestiona las matrices: físicamente, el hardware almacena los elementos de forma contigua, en una sola dimensión lineal. El compilador calcula la dirección de memoria exacta de cualquier coordenada empleando la fórmula de mapeo lineal, la cual depende estrictamente del número de columnas definido para cada fila.

Objetivo

Objetivo General

Reestructurar el código base del videojuego de simulación naval empleando el paradigma de diseño modular para delegar responsabilidades algorítmicas en subrutinas especializadas, implementando de forma correcta el pasaje de arreglos bidimensionales por referencia.

Objetivos Particulares

- Diseñar prototipos y firmas de función que acepten matrices de 5x5 enteros, respetando las restricciones sintácticas del lenguaje C.
- Separar la lógica de inicialización, posicionamiento de barcos, renderizado de mapas y evaluación de colisiones en funciones desacopladas.
- Demostrar experimentalmente cómo el pasaje por referencia nativo de los arreglos en C permite la modificación directa del estado de la matriz desde ámbitos externos al bloque principal.

Desarrollo

La metodología consistió en extraer los bloques secuenciales repetitivos e interactivos del código original y empaquetarlos en cuatro subrutinas bien definidas. A continuación se describe la implementación técnica del software modularizado:

```
#include <stdio.h>
#include <string.h>

// Prototipos globales con dimensionamiento de columnas mandatorio
void inicializarTablero(int t[][5]);
void colocarBarcos(int t[][5]);
void mostrarTablero(int t[][5]);
int procesarDisparo(int t[][5], int f, int c);

int main() {
    int tablero[5][5];
    int fila, columna;
    char jugador[20];

    printf("Nombre: ");
    fgets(jugador, sizeof(jugador), stdin);

    int longitud = strlen(jugador);
    if (longitud > 0 && jugador[longitud - 1] == '\n') {
        jugador[longitud - 1] = '\0';
    }

    // Invocación modular de los componentes del estado del juego
    inicializarTablero(tablero);
    colocarBarcos(tablero);

    printf("\n--- ¡BATALLA NAVAL INICIADA! ---\n");
    mostrarTablero(tablero);

    for(int turno = 0; turno < 5; turno++) {
        printf("\nTurno %d de %s -> ", turno + 1, jugador);
        printf("Fila (0-4): ");
        scanf("%d", &fila);
        printf("Columna (0-4): ");
        scanf("%d", &columna);
```

```

        if (fila < 0 || fila >= 5 || columna < 0 || columna >= 5) {
            printf("Coordenadas inválidas. Perdiste el tiro.\n");
            continue;
        }

        procesarDisparo(tablero, fila, columna);
        printf("\nMapa de impactos actual:\n");
        mostrarTablero(tablero);
    }

    printf("\nJuego terminado. ¡Buen intento, %s!\n", jugador);
    return 0;
}

void inicializarTablero(int t[][5]) {
    for(int i = 0; i < 5; i++) {
        for(int j = 0; j < 5; j++) {
            t[i][j] = 0;
        }
    }
}

void colocarBarcos(int t[][5]) {
    t[1][2] = 1;
    t[3][4] = 1;
    t[0][0] = 1;
}

void mostrarTablero(int t[][5]) {
    for(int i = 0; i < 5; i++) {
        for(int j = 0; j < 5; j++) {
            if(t[i][j] == 0 || t[i][j] == 1) {
                printf("~ ");
            } else if(t[i][j] == 2) {
                printf("o ");
            } else if(t[i][j] == 3) {
                printf("X ");
            }
        }
        printf("\n");
    }
}

int procesarDisparo(int t[][5], int f, int c) {
    if (t[f][c] == 3 || t[f][c] == 2) {
        printf("¡Ya habías disparado aquí!\n");
        return 0;
    }
    else if (t[f][c] == 1) {
        printf("¡IMPACTO!\n");
        t[f][c] = 3;
        return 1;
    }
    else {
        printf("Agua...\n");
        t[f][c] = 2;
        return 0;
    }
}

```

```
}  
}
```

Conclusión

La realización de esta práctica de laboratorio representó un avance cualitativo fundamental en mi comprensión de la ingeniería de software bajo el estándar de C. Lograr aislar los componentes interactivos del motor de juego de Batalla Naval y transformarlos en funciones estructuradas me demostró las ventajas de legibilidad y mantenibilidad que ofrece el código limpio.

El aspecto técnico más revelador de la sesión fue profundizar en el mapeo físico de la memoria RAM. Entender el motivo exacto por el cual la sintaxis de C obliga a definir explícitamente el tamaño de las columnas (`int t[][5]`) disipó dudas críticas sobre cómo el compilador traduce estructuras multidimensionales abstractas en secuencias contiguas de bytes. Asimismo, asimilar que las matrices se transfieren por referencia de manera predeterminada me otorga una ventaja competitiva de diseño, permitiéndome optimizar el uso de memoria en futuros algoritmos más complejos sin la necesidad de duplicar estructuras de datos pesadas.

Referencias

- Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.
- Deitel, P., & Deitel, H. (2016). *C How to Program* (8th ed.). Pearson.